



Tema 4.

Corpus y colecciones documentales extensas

Organización y catalogación a gran escala

Material teórico | Datos Masivos | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

ÍNDICE DE CONTENIDOS	1
CONTEXTO PROFESIONAL	1
QUÉ VAS A APRENDER EN ESTA UNIDAD	1
MAPA CONCEPTUAL DE LA UNIDAD	1
1. MARCO TEÓRICO	2
1.1. De la colección de archivos al corpus	2
1.2. Por qué catalogar antes de analizar	2
1.3. Organización de archivos y carpetas a escala	2
1.4. Recorrer una colección con Python	2
1.5. Extraer metadatos de cada documento	3
1.6. Construir el catálogo con Pandas	3
1.7. De archivos sueltos a tabla: la misma lógica de la Unidad 2	4
2. APLICACIÓN PRÁCTICA	4
2.1. Aplicación al caso de la Fundación Orellana	4
2.2. Bloque final opcional: una colección real y grande	4
3. QUÉ QUEDA FUERA DE ESTA UNIDAD, Y POR QUÉ	5
BIBLIOGRAFÍA	5

Contexto profesional

Imagina que acabas de incorporarte como analista junior de datos documentales a **Corpus & Contexto**, una consultora ficticia especializada en organización y explotación de colecciones documentales. Tu cliente es la **Fundación Orellana**, una entidad ficticia que custodia el legado de un escritor del siglo XX: cartas, manuscritos y recortes de prensa relacionados con su obra.

Durante los últimos años, un equipo de voluntariado ha ido digitalizando y pasando por reconocimiento óptico de caracteres (OCR) miles de páginas de ese legado, guardando cada documento como un archivo de texto suelto, organizado de forma irregular en carpetas por tipo de material. Nadie en la Fundación sabe con precisión cuántos documentos hay digitalizados, cómo se reparten entre cartas, manuscritos y prensa, ni qué volumen de texto representan en conjunto.

Antes de plantearse cualquier estudio filológico o computacional sobre el legado, la Fundación necesita algo mucho más básico: un catálogo fiable de lo que ya tiene. Ese catálogo es tu encargo.

Qué vas a aprender en esta unidad

Al terminar esta unidad deberías ser capaz de:

- explicar qué distingue un corpus de una simple colección de archivos;
- recorrer una estructura de carpetas con Python para localizar documentos;
- extraer metadatos básicos de cada archivo sin leer su contenido en profundidad;
- construir con Pandas una tabla-catálogo a partir de muchos archivos sueltos;
- exportar y documentar un catálogo de una colección documental extensa.

Mapa conceptual de la unidad

```
Colección de archivos sueltos
-> recorrer carpetas (os, pathlib)
-> extraer metadatos por documento
-> construir tabla con Pandas
-> agregados por subcarpeta
-> exportación del catálogo
-> documentación de decisiones
```

Como en la Unidad 2, la idea central es que no se puede analizar lo que no se ha inventariado antes. Aquí, en vez de partir de un único archivo con muchos registros, se parte de muchos archivos con un registro cada uno. El objetivo es el mismo: llegar a una tabla clara, reproducible y documentada.

1. Marco teórico

1.1. De la colección de archivos al corpus

No cualquier montón de archivos de texto es un corpus. McEnery y Hardie (2012) señalan que lo que distingue a un corpus de una colección arbitraria de textos es que se reúne de forma **principiada**: con un criterio explícito sobre qué se incluye, de dónde procede y cómo está organizado, de manera que otra persona pueda entender y reproducir esos criterios. Una carpeta con miles de archivos escaneados sin catalogar, por muy valiosa que sea la colección subyacente, todavía no es un corpus en ese sentido: es una colección pendiente de convertirse en uno.

El primer paso hacia ese criterio explícito no es analizar el contenido, sino inventariarlo: saber qué hay, cuánto hay y cómo está organizado. Esa fase de inventario es la que trabaja esta unidad.

1.2. Por qué catalogar antes de analizar

El mismo argumento que ya se trabajó en la Unidad 1 y la Unidad 2 se aplica aquí, con una escala distinta. Kandel *et al.* (2012) documentan que gran parte del tiempo real de un proyecto de datos se dedica a localizar, describir y estructurar la información antes de que empiece cualquier análisis. Cuando el material no es una tabla, sino miles de archivos sueltos, esa fase de localización y descripción se vuelve todavía más necesaria: sin un catálogo, ni siquiera es posible plantear cuántos documentos hay disponibles para un estudio futuro.

En el caso de la Fundación Orellana, el catálogo que vas a construir no interpretará ni una sola carta ni un solo recorte de prensa. Se limita a responder preguntas de inventario: cuántos documentos hay, dónde están, qué tamaño tienen y, de forma aproximada, cuánto texto contienen.

1.3. Organización de archivos y carpetas a escala

Cuando una colección crece hasta miles de archivos, la forma en que se organizan las carpetas y se nombran los archivos deja de ser un detalle estético y se convierte en una condición de trabajo. Wilson *et al.* (2017), en una guía de buenas prácticas de computación científica ampliamente adoptada, recomiendan mantener siempre una separación clara entre los datos brutos y cualquier dato derivado o procesado, no modificar nunca los datos brutos directamente, y usar nombres de archivo y de carpeta uniformes e informativos en vez de nombres genéricos o ambiguos.

Esta recomendación conecta directamente con una distinción que ya trabajaste en la Unidad 2: archivo bruto, versión de trabajo y versión preparada. Aquí se aplica a nivel de colección completa: la carpeta ``coleccion_orellana/`` con los archivos digitalizados originales es el material bruto y no se modifica; el catálogo que vas a construir es el producto derivado, documentado y exportado aparte.

1.4. Recorrer una colección con Python

Para inventariar una colección de archivos hace falta poder recorrer una estructura de carpetas de forma programática. Python ofrece para esto el módulo estándar ``pathlib``, que representa rutas de

archivo como objetos manipulables. Un patrón habitual es usar el método ``rglob()`` de un objeto ``Path`` para encontrar, de forma recursiva, todos los archivos que cumplen un patrón (por ejemplo, todos los ``.txt`` dentro de una carpeta y sus subcarpetas).

Cada archivo encontrado de esta forma expone metadatos básicos sin necesidad de abrirlo: su nombre, la carpeta que lo contiene y su tamaño en bytes (a través de ``stat().st_size``). Estos metadatos son exactamente lo que un catálogo de inventario necesita como punto de partida.

1.5. Extraer metadatos de cada documento

Además del nombre, la ruta y el tamaño, un catálogo documental suele necesitar una estimación del volumen de texto que contiene cada archivo. No hace falta leer el documento con atención para eso: basta con abrirlo, dividir su contenido por espacios en blanco y contar cuántos elementos resultan. Esta operación no interpreta el texto: es una transformación descriptiva, del mismo tipo que las que ya se trabajaron en la Unidad 2 (contar menciones, contar URL) aplicada ahora a nivel de documento completo en vez de a nivel de columna.

Es importante no confundir esta cuenta aproximada de palabras con un análisis lingüístico. Contar cuántas palabras tiene una carta no dice nada sobre lo que la carta dice.

1.6. Construir el catálogo con Pandas

Una vez que se dispone, para cada documento, de su nombre, su carpeta de procedencia, su tamaño y su recuento aproximado de palabras, el paso siguiente es exactamente el que ya se trabajó en la Unidad 2: convertir una colección de registros (en este caso, uno por archivo) en un ``dataframe`` de Pandas. La diferencia es el origen del registro: en la Unidad 2 cada registro venía de un elemento de una lista JSON ya cargada en memoria; aquí, cada registro se construye explorando el sistema de archivos, uno por documento.

El resultado es una tabla con una fila por documento, lista para agregarse (por ejemplo, número de documentos y palabras totales por subcarpeta con ``groupby()``), para filtrarse o para exportarse, igual que cualquier otro ``dataframe`` preparado en esta asignatura.



Figura 1. De la colección de archivos sueltos al catálogo con Pandas (elaboración propia).

1.7. De archivos sueltos a tabla: la misma lógica de la Unidad 2

Conviene ser explícito sobre lo que cambia y lo que no cambia respecto a la Unidad 2. Lo que cambia es el origen de los datos: un único archivo JSON con muchos registros frente a muchos archivos de texto con un documento cada uno. Lo que no cambia es la lógica de preparación: inspeccionar antes de actuar, seleccionar solo la información relevante para el encargo, documentar las decisiones y exportar un producto revisable. Reconocer esta continuidad es tan importante como aprender la técnica concreta de recorrido de carpetas: la próxima vez que te encuentres una colección de archivos en vez de una tabla, sabrás que el problema de fondo es el mismo.

2. Aplicación práctica

2.1. Aplicación al caso de la Fundación Orellana

En el cuaderno de clase seguirás este recorrido:

1. recorrer la carpeta ``coleccion_orellana/`` y localizar todos los documentos ``*.txt``;
2. extraer, para cada uno, su nombre, su subcarpeta, su tamaño y su recuento aproximado de palabras;
3. construir un ``dataframe`` con un registro por documento;
4. calcular agregados por subcarpeta (número de documentos y palabras totales);
5. exportar el catálogo como CSV, documentando las decisiones tomadas.

El resultado no será ninguna valoración sobre el contenido del legado de la Fundación Orellana. Será un inventario que permita, en fases posteriores fuera de esta unidad, decidir con criterio qué parte de la colección merece un estudio más detallado.

2.2. Bloque final opcional: una colección real y grande

Al final del cuaderno de clase hay un bloque opcional, que no se evalúa, en el que se compara la colección sintética de la Fundación Orellana con una colección real: 1.202 libros en español de dominio público del proyecto Project Gutenberg, obtenidos a través de la librería Hugging Face datasets. El objetivo es comprobar algo que ya se apunta en la Unidad 1: el método de trabajo no cambia con la escala, pero la experiencia de trabajar con datos reales sí cambia — en tiempo de ejecución, en memoria y en la aparición de problemas que una colección sintética bien construida no tiene, como los registros duplicados.

Para que la sesión de clase no dependa de que todo el grupo descargue a la vez varios cientos de megabytes por la misma conexión wifi, la descarga se hace como tarea previa, igual que la lectura de este manual: antes de la sesión del cuaderno principal, con el entorno virtual `.venv` ya activado, cada alumno ejecuta una vez el código de carga del *dataset* (conjunto de datos) (indicado en el propio cuaderno de clase). La librería datasets guarda automáticamente una copia local la primera vez que se ejecuta, en una carpeta de caché que gestiona ella misma; no hace falta descargar manualmente ningún archivo ni indicar ninguna ruta, y el mismo código funciona igual en Windows, macOS o Linux, tanto en un ordenador personal como en uno del aula.

3. Qué queda fuera de esta unidad, y por qué

Por qué no se lee ni se interpreta el contenido de ningún documento más allá del recuento aproximado de palabras, ni se corrige el texto reconocido por OCR. El objetivo de esta unidad es catalogar la colección, no analizarla: son dos tareas metodológicamente distintas, y mezclarlas invalidaría el inventario que estás construyendo. Corregir el texto de un OCR, en concreto, exigiría comparar el texto reconocido con el documento original página a página, una tarea de revisión filológica que queda fuera del alcance de un catálogo a escala como el de esta unidad.

Por qué no se extraen conclusiones sobre el estilo o el valor literario del legado. Valorar el estilo o el interés literario de una colección exige herramientas de crítica textual y estudios literarios que no se cubren ni en esta asignatura ni en Programación orientada a PLN: no es que lo vayas a ver más adelante, es que sencillamente no forma parte del recorrido de ningún curso de este grado tal y como está diseñado ahora mismo.

Por qué no se extraen conclusiones sobre los temas del legado. Esto sí conecta con una técnica que ya cursaste: si algún día quisieras explorar de qué tratan los documentos de una colección como esta, sin leerlos uno a uno, la unidad sobre clasificación temática de textos de Programación orientada a PLN enseña exactamente eso — representar cada documento como un vector de palabras y compararlo con vectores prototipo de cada tema, o entrenar un clasificador sobre ejemplos ya etiquetados. Con el catálogo que construyas en esta unidad como punto de partida (nombre de archivo, subcarpeta, tamaño, recuento de palabras), si lo desearas podrías cargar después el texto completo de cada documento y aplicar esa misma técnica para explorar sus temas, tal y como se narra en esa unidad de la asignatura anterior. Aquí no lo hacemos porque catalogar y clasificar son dos fases distintas del mismo proyecto, y esta unidad se ocupa solo de la primera.

Por qué no se inventa ningún documento, fecha o autoría que no conste en los propios archivos de la colección. Esta norma se aplica a toda la asignatura: cualquier metadato que uses debe poder rastrearse hasta un archivo concreto de la colección. Para una institución real como la Fundación Orellana del caso, atribuir una fecha o una autoría inventada a un documento histórico tendría consecuencias serias, no solo académicas.

Bibliografía

Hugging Face. (s. f.). manu/project_gutenberg [Conjunto de datos]. Recuperado el 28 de julio de 2026 de https://huggingface.co/datasets/manu/project_gutenberg

Kandel, S., Paepcke, A., Hellerstein, J. M. y Heer, J. (2012). Enterprise data analysis and visualization: An interview study. *IEEE Transactions on Visualization and Computer Graphics*, 18(12), 2917-2926. <https://doi.org/10.1109/TVCG.2012.219>

McEnery, T. y Hardie, A. (2012). *Corpus linguistics: Method, theory and practice*. Cambridge University Press.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>

The pandas development team. (2020). *pandas-dev/pandas: Pandas* [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

Project Gutenberg. (s. f.). Project Gutenberg [Colección digital]. Recuperado el 28 de julio de 2026 de <https://www.gutenberg.org/>

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L. y Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>